R23: III B.Tech I Semester

FORMAL LANGUAGES AND AUTOMATA THEORY

UNIT-I

Finite Automata: Need of Automata theory, Central Concepts of Automata Theory, Automation, Finite Automation, Transition Systems, Acceptance of a String, DFA, Design of DFAs, NFA, Design of NFA, Equivalence of DFA and NFA, Conversion of NFA into DFA, Finite Automata with Є-Transitions, Minimization of Finite Automata, Finite Automata with output-Mealy and Moore Machines, Applications and Limitation of Finite Automata.

1.1 Need of Automata theory

- Automata theory, also known as the Theory of Computation, is a field within computer science and mathematics that focuses on studying abstract machines to understand the capabilities and limitations of computation by analyzing mathematical models of how machines can perform calculations.
- Automata theory is a fundamental area of computer science, with real-world applications in various systems and domains:

(i) Regular Expressions (RE) in Systems

- Regular expressions are powerful tools for **pattern matching and text processing** used extensively in many systems.
- Examples:
  - **UNIX:** In UNIX, regular expressions like a.*b are used for matching text patterns within files, making it easier to search for specific content across vast datasets.
  - **XML and DTDs:** Document Type Definitions (DTDs) describe the structure of XML documents using regular expressions. For example, a tag like person (name, addr, child*) ensures that the person tag must include a name, an addr, and optionally multiple child tags.
  - **Programming Languages:** Almost all modern programming languages have libraries for regular expression that allow us to do text processing.

(ii) Finite Automata in Modeling Systems

- **Modeling Protocols and Circuits**: Finite automata (FA) are used to model protocols, like those in network communication, and to design electronic circuits that operate based on a set of predefined rules or states.
- **Model-Checking**: FA theory is also applied in model-checking, which is used to verify whether a system behaves as expected under all possible conditions.

(iii) Context-Free Grammars (CFG)

- **Syntax of Programming Languages**: Context-free grammars are essential in describing the syntax of most programming languages. They define the rules that specify how programs should be written and structured.
- **Natural Language Processing**: CFGs also play a vital role in computational linguistics, helping to describe the structure of natural languages like English.
- **XML and DTDs as CFGs**: DTDs (Document Type Definitions) can be thought of as a specific application of context-free grammars, as they define the structure of XML documents.

1.2 Central Concepts of Automata Theory

1.2.1 Major areas of Computation Theory

1. Automata Theory

- Automata theory studies abstract computational models and their applications. It forms the basis for understanding how machines process inputs and produce outputs. Key components include:
  - **Finite Automata**: Used to model simple systems like lexical analyzers in compilers.
  - **Pushdown Automata**: A more powerful model capable of recognizing context-free languages, essential for parsing programming languages.
  - **Turing Machines**: The most powerful automata, used as a standard for defining what is computable.

2. Formal Languages and Grammars

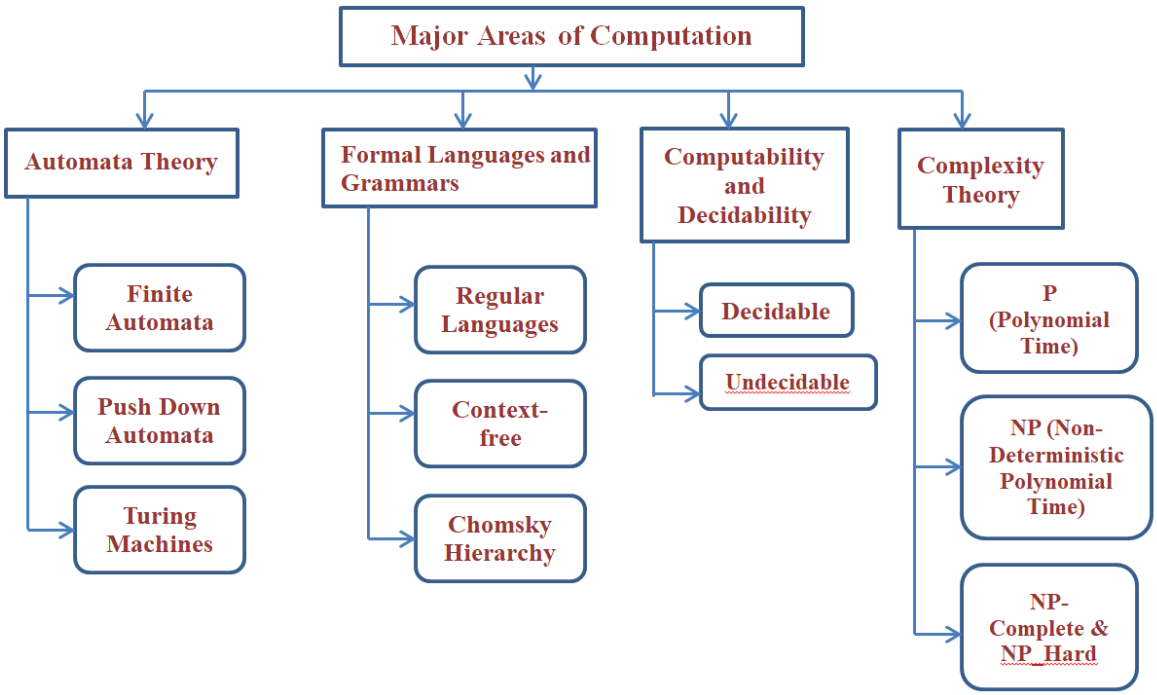
- This area examines the syntax and structure of languages used in computation. It involves:
  - **Regular Languages:** Described by regular expressions and finite automata, representing simple patterns.
  - **Context-Free Languages:** Defined by context-free grammars, crucial for designing compilers.
  - **Chomsky Hierarchy:** A classification of languages into regular, context-free, context-sensitive, and recursively enumerable languages.

3. Computability and Decidability

- Computability theory addresses the question: What problems can a computer solve? It studies concepts like:
  - **Decidable Problems:** Problems with an algorithmic solution.
  - **Undecidable Problems:** Problems, such as the Halting Problem, for which no algorithm can determine a solution for all inputs.

4. Complexity Theory

- Complexity theory focuses on the efficiency of algorithms by analyzing the time and space resources they require. It categorizes problems into classes such as:
  - **P (Polynomial Time):** Problems solvable in polynomial time.
  - **NP (Nondeterministic Polynomial Time):** Problems whose solutions can be verified in polynomial time.
  - **NP-Complete and NP-Hard:** The most challenging problems in NP, with applications in cryptography, optimization, and artificial intelligence.



1.2.2 Alphabet

An alphabet is a finite, nonempty set of symbols. It is denoted by ∑.

Example:

∑= {0, 1} is binary alphabet consisting of the symbols 0 and 1.

∑= {a, b, c ...z} is lowercase English alphabet.

1.2.3 Powers of an Alphabet

If Σ is an alphabet, we can express the set of all strings of a certain length from that alphabet by using the exponential notation. It is denoted by Σ^k - the set of strings of length k.

Example:

Σ^0 = {ε}, regardless of what alphabet Σ is. ε is the only string of length 0.

If Σ = {0, 1} then,

Σ^1 = {0, 1}

Σ^2 = {00, 01, 10, 11}

Σ^3 = {000, 001, 010, 011, 100, 101, 110, 111}

The set of all strings over an alphabet Σ is denoted by Σ*. $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$

For example, {0, 1}* = {ε, 0, 1, 00, 01, 10, 11, 000, ...}

The symbol ∗ is called ***Kleene star*** and is named after the mathematician and logician Stephen Cole Kleene.

The symbol + is called ***Positive closure*** i.e. $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \ldots$

$$\Sigma^* = \Sigma^+ \cup \{ \varepsilon \}$$

1.2.4 String:

A string (or word) is a finite sequence of symbols chosen from some alphabet.

The letters u, v, w, x, y and z are used to denote string.

Example:

If Σ = {a, b, c} then abcb is a string formed from that alphabet.

- The ***length*** of a string w, denoted |**w**|, is the number of symbols composing the string.

Example:

The string abcb has length 4.

- The ***empty string*** denoted by **ε**, is the string consisting of zero symbols.
  Thus |ε| =0.

1.2.5 Operations on strings:

- ***Concatenation of strings:*** The concatenation of two strings is the string formed by writing the first, followed by the second, with no intervening space. Concatenation of strings is denoted by ◦.That is, if w and x are strings, then wx is the concatenation of these two strings.
  Example:
  The concatenation of dog and house is doghouse.
  Let x=0100101 and y= 1111 then x ◦ y=01001011111
- ***String Reversal:*** Reversing a string means writing the string backwards. It is denoted by w^R
  Example:
  Reverse of the string abcd is dcba.
  If $w= w^R$, then that string is called palindrome.
- ***Substring:*** A substring is a part of a string.
  Example:
  If abcd is string then possible substrings are **ε**,a,b,c,d,ab,bc,cd,abc,bcd are proper substrings for the given string
  A ***prefix*** of a string is any number of leading symbols of that string.
  A ***suffix*** of a string is any number of trailing symbols.
  Example:
  String abc has prefixes **ε**, a, ab, and abc; its suffixes are **ε**, c, bc, and abc.
  A prefix or suffix of a string, other than the string itself, is called a ***proper prefix or suffix***.

1.2.6 Language:

A language is a set of strings of symbols from some alphabet. It is denoted by L. We denote this language by ∑*.

- The empty set, Ø, and the set consisting of the empty string {**ε**} are languages.

Example:

If ∑= {a}, then ∑* = {**ε**, a, aa, aaa, ...}.
If ∑ = {0, 1}, then ∑* = {**ε**, 0, 1, 00, 01, 10, 11, 000,...}.

1.2.7 Operations on languages:

- ***Union***
  If L1 and L2 are two languages over an alphabet ∑.Then the union of L1 and L2 is denoted by L1 U L2.
  Example:
  L1={0,01,011} and L2={001},then L1 U L2={0,01,011,001}
- ***Intersection***
  If L1 and L2 are two languages over an alphabet ∑.Then the intersection of L1 and L2 is denoted by L1 ∩ L2.
  Example:
  L1= {0, 01,011} and L2= {01}, then L1 ∩ L2= {01}
- ***Complementation***
  L is a language over an alphabet ∑, then the complement of L denoted by L' ,is the language consisting of those strings that are not in L over the alphabet.
  Example:
  If ∑={a,b} and L={a,b,aa} then

L'=∑*-L={**ε,a,b,aa,bb,ab.....}-{a,b,aa}={ε,bb,ab,ba.....}**

- ***Concatenation***
  Concatenation of two languages L1 and L2 is the language L1 o L2 ,each element of which is a string formed by combining one string of L1 with another string of L2.
  Example:
  L1={bc,bcc,cc}andL2={cc,ccc},thenL1oL2={bccc,bcccc,bccccc,cccc,ccccc}
- ***Reversal***
  If L is language, then L^R is obtained by reversing the corresponding string in L. This operation is similar to the reversal of a string.

$$L^R = \{w^R \mid w \in L\}$$

Example:
If L= {0, 011, 0111}, then L^R = {0, 110, 1110}

- ***Kleene Closure***
  The Kleene closure (or just closure) of L, denoted L*, is the set

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

and the positive closure of L, denoted L^+ , is the set

$$L^+ = \bigcup_{i=1}^{\infty} L^i$$

L* denotes words constructed by concatenating any number of words from L.
L+ is the same, but the case of zero words, whose "concatenation" is defined to be **ε**, is excluded.
Note that L+ contains **ε** if and only if L does.

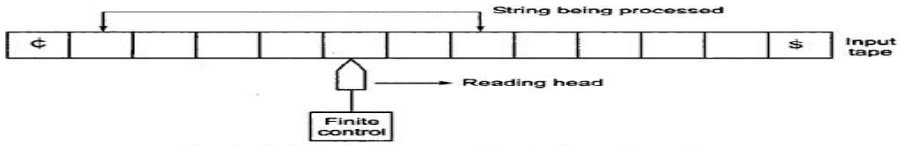
Example:
Let L1 = {10, 1}
Then **L *** = L0 U L1 U L2..... = {ε, 1, 10, 11, 111, 1111 ,.....}
L^+ = L1 U L2 U L3.....= {1, 10, 11, 111, 1111.....}

1.3 Finite Automaton:

- A finite automaton (FA) consists of a finite set of states and a set of transitions from state to state that occur on input symbols chosen from an alphabet ∑.
- For each input symbol there is exactly one transition out of each state (possibly back to the state itself).
- One state, usually denoted q_0 is the initial state, in which the automaton starts. Some states are designated as final or accepting states.
  Formally, a finite automaton is denoted by a 5-tuple **(Q, ∑, δ, q_0, F),** where
  - Q is a finite set of states.
  - ∑ is a finite input alphabet.
  - **δ** is the transition function mapping Q x ∑ to Q i.e., **δ** (q,a) is a state for each state q and input symbol a.
  - qo ∈ Q is the initial state.

- F ⊆ Q is the set of final states. It is assumed here that there may be
- more than one final state.

1.4 Finite Automata Model:



Block diagram of a finite automaton

The various components are explained as follows:

(i) ***Input tape:***

- The input tape is divided into squares, each square containing a single symbol from the input alphabet ∑.
- The end squares of the tape contain the endmarker ¢ at the left end and the endmarker $ at the right end.
- The absence of endmarkers indicates that the tape is of infinite length. The left-to-right sequence of symbols between the two endmarkers is the input string to be processed.

(ii) ***Reading head:***

- The head examines only one square at a time and can move one square either to the left or to the right.
- For further analysis, we restrict the movement of the R-head only to the right side.

(iii) ***Finite control:*** The input to the finite control will usually be the symbol under the R-head, say a, and the present state of the machine, say q, to give the following outputs:

(a) A motion of R-head along the tape to the next square (in some a null move, i.e. the R-head remaining to the same square is permitted)

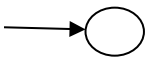
(b) The next state of the finite state machine given by δ(q, a).

1.5 Transition Diagram:

- A transition diagram is a directed graph associated with an FA in which the vertices of the graph correspond to the states of the FA.
- If there is a transition from state q to state p on input a, then there is an arc labelled a from state q to state p in the transition diagram.

State is denoted by 

Transition is denoted by 

Initial state is denoted by 

Final state is denoted by 

1.5.1 Transition Table:

A tabular representation in which rows correspond to states, columns correspond to inputs and entries correspond to next states.

1.6 Acceptance of Strings by a Finite Automaton:

The FA accepts a string x if the sequence of transitions corresponding to the symbols of x leads from the start state to an accepting state and the entire string has to be consumed, i.e., a string x is accepted by a finite automaton M = **(Q, ∑, δ, q_o, F)**

$$\text{if } \delta(q_0, x) = q \text{ for some } q \in F.$$

This is basically the acceptability of a string by the final state.

Note: A final state is also called an accepting state.

Transition function δ and for any two input strings x and y,

$$\delta(q, xy) = \delta(\delta(q, x), y)$$

Example:

Consider the finite state machine whose transition function δ is given in the form of a transition table. Here Q = {q_0, q_1, q_2, q_3},∑={0,1}, F={q_0}.Give the entire sequence of states for the input string 110101.

Transition Table

State	Input	
	0	**1**
→ q_0	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

$\delta(q_0, 110101) = \delta(q_1, 10101)$

$= \delta(q_0, 0101)$

$= \delta(q_2, 101)$

$= \delta(q_3, 01)$

$= \delta(q_2, 1)$

$= q0$

q_0 is final state, therefore given string is accepted by finite automata.

Note: The language accepted by Finite Automata M designated L(M) is the set {x | δ(q0,x) is in F}.

1.7 Deterministic finite automaton:

Formally, a deterministic finite automaton can be represented by a 5-tuple, M= **(Q, ∑, δ, q_o, F)** where

- Q is a finite set of states.
- ∑ is a finite input alphabet.
- **δ** is the transition function mapping Q x ∑ to Q i.e., **δ** (q,a) is a state for each state q and input symbol a.
- q_o ∈ Q is the initial state.

- $F \subseteq Q$ is the set of final states.

Steps to design a DFA

1. Understand the language for which the DFA has to be designed and write the language for the set of strings starting with minimum string that are accepted by FA.
2. Draw transition diagram for the minimum length string.
3. Obtain the possible transitions to be made for each state on each input symbol.
4. Draw the transition table.
5. Test DFA with few strings that are accepted and few strings that are rejected by the given language.
6. Represent DFA with tuples.

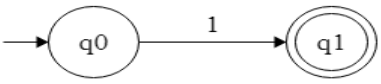
Examples

1. Design DFA that accepts all strings which starts with '1' over the alphabet {0,1}

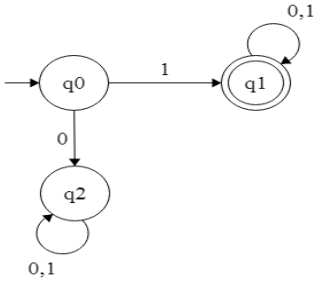
Step 1: Understand the language for which the DFA has to be designed and write the language for the set of strings starting with minimum string that is accepted by FA.

L = {1, 10, 11, 100, 110, 101, 111, ...}

Step 2: Draw transition diagram for the minimum length string.



Step 3: Obtain the possible transitions to be made for each state on each input symbol.



Step 4: Draw the transition table.

Stat	Input	
	0	**1**
q_0	q_2	q_1
q	q_1	q_1
q_2	q_2	q_2

Step 5: Test DFA with few strings that are accepted and few strings that are rejected by the given language.

Case i) Let w=1001 ∈ L

$\delta(q_0,1001) = \delta(q1,010)$

$= \delta(q_1,10) = \delta(q_1,0) = q_1$

q_1 is final state and the entire string has been consumed i.e., given string is accepted by DFA.

Case ii) Let w=0001 ∉ L

$\delta(q0,0001) = \delta(q2,001)$

$= \delta(q2,10)$

$= \delta(q2,0)$

$= q_2$

q_2 is not final state and the entire string has been consumed i.e., given string is rejected by DFA.

Step 6: Represent DFA with tuples.

DFA, M= **(Q, ∑, δ, qo, F)**

where Q = $\{q_0, q_1, q_2\}$

∑ = { 0,1 }

δ: $\delta(q_0,0)=q_2$

$\delta(q_0,1)=q_1$

$\delta(q_1,0)=q_1$

$\delta(q_1,1)=q_1$

$\delta(q_2,0)=q_2$

$\delta(q_2,1)=q_2$

q_0 – initial state

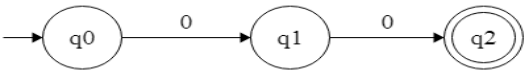
F – final state = $\{ q_1\}$

2. Design DFA that accepts all strings which contains '00' as substring over the alphabet {0,1}

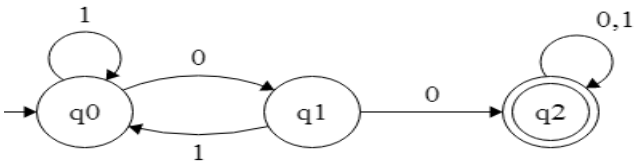
Step 1: Understand the language for which the DFA has to be designed and write the language for the set of strings starting with minimum string that is accepted by FA.

L={00,100,000,001,1100,1000,0100,1001,0001,11000,11100,.....}

Step 2: Draw transition diagram for the minimum length string.



Step 3 : Obtain the possible transitions to be made for each state on each input symbol.



Step 4: Draw the transition table.

State	Input	
	0	**1**
→ q_0	q_1	q_0
q_1	q_2	q_0
(q)	q_2	q_2

Step 5: Test DFA with few strings that are accepted and few strings that are rejected by the given language.

Case i) Let w = 1001 ∈ L

$\delta(q_0,1001) = \delta(q_0,001)$

$= \delta(q_1,01)$

$= \delta(q_2,1)$

$= q_2$

q2 is final state and the entire string has been consumed i.e., given string is accepted by DFA.

Case ii) Let w=1011 $\notin$ L

$\delta(q_0,1011) = \delta(q_0,011)$
$= \delta(q_1,11)$
$= \delta(q_0,1)$
$= q_0$

q0 is not final state and the entire string has been consumed i.e., given string is rejected by DFA.

Step 6: Represent DFA with tuples.

DFA, M= **(Q, ∑, δ, qo, F)**
where Q = $\{q_0, q_1, q_2\}$
∑ = { 0,1 }
δ: $\delta(q_0,0)=q_1$
$\delta(q_0,1)=q_0$
$\delta(q_1,0)=q_2$
$\delta(q_1,1)=q_0$
$\delta(q_2,0)=q_2$
$\delta(q_2,1)=q_2$
q_0 – initial state
F – final state = { q_2 }

1.8 Non-deterministic finite automaton (NDFA/NFA):

A nondeterministic finite automaton is a 5-tuple (Q, ∑, δ, qo, F), where

- Q is a finite nonempty set of states;
- ∑ is a finite nonempty set of inputs;
- δ is the transition function mapping from Q x ∑ into 2^Q which is the power set of Q, the set of all subsets of Q;
- qo ∈ Q is the initial state; and
- F ⊆ Q is the set of final states

Steps to design a NFA

1. Understand the language for which the NFA has to be designed and write the language for the set of strings starting with minimum string that is accepted by FA.
2. Draw transition diagram for the minimum length string.
3. Obtain the possible transitions to be made for each state on each input symbol.
4. Draw the transition table.
5. Test NFA with few strings that are accepted and few strings that are rejected by the given language.
6. Represent NFA with tuples.

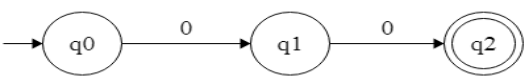
Examples:

1. **Design NFA that accepts all strings which contains '00' as substring over the alphabet {0,1}**

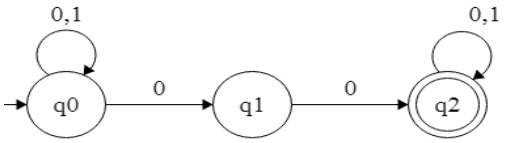
Step 1: Understand the language for which the NFA has to be designed and write the language for the set of strings starting with minimum string that is accepted by FA

L={00,100,000,001,0100,1100,1000,1001,0001,11000,11100,.....}

Step 2: Draw transition diagram for the minimum length string.



Step 3: Obtain the possible transitions to be made for each state on each input symbol.



Step 4: Draw the transition table.

State	Input	
	0	**1**
→ q_0	$\{q_0,q_1\}$	q_0
q	q_2	-
q_2	q_2	q_2

Step 5: Test NFA with few strings that are accepted and few strings that are rejected by the given language.

Case i) Let w=0100 ∈ L

$\delta(q_0,0100) = \delta(\{q_0,q_1\},100)$

$= \delta(q_0, 00)$

$= \delta(\{q_0, q1\},0)$

$= \{q_0, q1, q2\}$

q_2 is final state and the entire string has been consumed i.e., given string is accepted by NFA.

Case ii) Let w=1011 ∉ L

$\delta(q_0,1011) = \delta(q_0,011)$

$= \delta(\{q_0,q_1\},11)$

$= \delta(q_0,1)$

$= q_0$

q_0 is not final state and so given string is rejected by NFA.

Step 6: Represent NFA with tuples.

NFA, M= **(Q, ∑, δ, qo, F)**

where Q = $\{q_0, q_1, q_2\}$

∑ = { 0,1 }

δ: $\delta(q_0,0)=\{q_0,q_1\}$

$\delta(q_0,1) = q_0$

$\delta(q_1,0) = q_2$

$\delta(q_1,1) = \emptyset$

$\delta(q_2,0) = q_2$

$\delta(q_2,1) = q_2$

q_0 – initial state

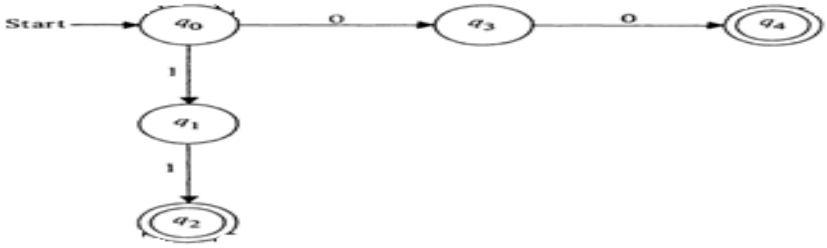
F – final state = $\{ q_2 \}$

2. **Design NFA that accepts strings which contains either two consecutive 0's or two consecutive 1's.**

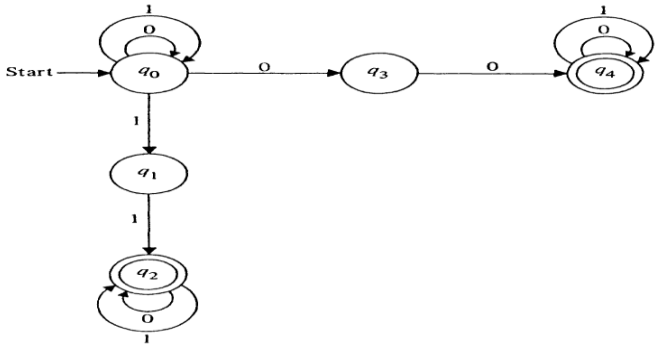
Step 1: Understand the language for which the NFA has to be designed and write the language for the set of strings starting with minimum string that is accepted by FA.

L={00,11,100,001,110,011,111,000,0100,1011,.....}

Step 2: Draw transition diagram for the minimum length string.



Step 3: Obtain the possible transitions to be made for each state on each input symbol.



Step 4: Draw the transition table.

State	Input	
	0	**1**
→ q_0	$\{q_0,q_3\}$	$\{q_0,q_1\}$
q_1	-	q_2
(a) q_2	q_2	q_2
q_3	q_4	-
(a) q_4	q_4	q_4

Step 5: Test NFA with few strings that are accepted and few strings that are rejected by the given language.

Case i) Let the input, w = 01001 ∈ L

$\delta(q_0,0) = \{q_0,q_3\}$

$\delta(q_0,01) = \delta(\delta(q_0,0),1) = \delta(\{q_0,q_3\},1) = \delta(q_0,1) \cup \delta(q_3,1) = \{q_0,q_1\}$

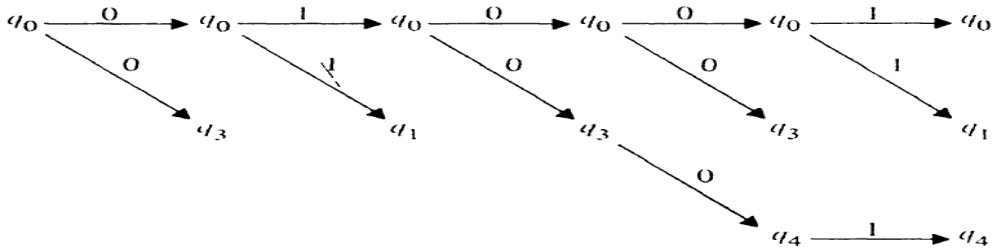
Similarly, we compute

$\delta(q_0,010) = \{q_0,q_3\}$, $\delta(q_0,0100) = \{q_0,q_3,q_4\}$

and

$\delta(q_0,01001) = \{q_0,q_1,q_4\}$

After the entire string is consumed, the FA is in state q_4.As q_4 is the final state, the string is a accepted by FA



Case ii) Let w = 010 ∉ L

$\delta(q_0,010) = \delta(\{q_0,q_3\},10)$

$= \delta(\{q_0,q_1\},0)$

$= \{q_0,q_3\}$

There is no path to the final state after the entire string is consumed. So the string is rejected by FA.

Step 6: Represent NFA with tuples.

NFA, M= **(Q, ∑, δ, qo, F)**

where Q = $\{q_0, q_1, q_2, q_3, q_4\}$

∑ = { 0,1 }

δ: $\delta(q_0,0)=\{q_0,q_3\}$

$\delta(q_0,1)= \{q_0,q_1\}$

$\delta(q_1,0)= Ø$

$\delta(q_1,1)=q_2$

$\delta(q_2,0)=q_2$

$\delta(q_2,1)=q_2$

$\delta(q_3,0)=q_4$

$\delta(q_3,1)= Ø$

$\delta(q_4,0)=q_4$

$\delta(q_4,1)=q_4$

q_0 – initial state

F – final state = { q_2,q_4 }

1.9 NFA to DFA Conversion:

Step 1: First take the starting state of NFA as the starting state of DFA.

Step 2: Apply the inputs on initial state and represent the corresponding states in the transition table.

Step 3: For each newly generated state, apply the inputs and represent the corresponding states in the transition table.

Step 4: Repeat step 3 until no more new states are generated.

Step 5: The states which contain any of the final states of the NFA are the final states of the equivalent DFA.

Step 6: Represent the transition diagram from the constructed table.

Step7: To check the equivalence of NFA and DFA, the string accepted by NFA should be accepted by DFA.

Step 8: Write the tuple representation for the obtained DFA.

Note: If the NFA has *n* states, the resulting DFA may have up to 2^n states, an exponentially larger number, which sometimes makes the construction impractical for large NFAs.

Example:

1. Construct DFA equivalent to the NFA M=($\{q_0,q_1\},\{0,1\}, \delta, q_0, \{q_1\}$)

where $\delta(q_0,0) = \{q_0,q_1\}$ $\delta(q_0,1) = \{q_1\}$ $\delta(q_1,0) = \emptyset$ $\delta(q_1,1) = \{q_0,q_1\}$

Step 1: First take the starting state of NFA as the starting state of DFA

Q/∑	0	1
→ [q_0]		

Step 2: Apply the inputs on initial state and represent the corresponding states in the transition table.

Q/∑	0	1
→ [q_0]	[q_0,q_1]	[q_1]

Step 3: For each newly generated state, apply the inputs and represent the corresponding states in the transition table.

Q/∑	0	1
→ [q_0]	[q_0,q_1]	[q_1]
[q_0,q_1]	[q_0,q_1]	[q_0,q_1]
[q_1]	Ø	[q_0,q_1]

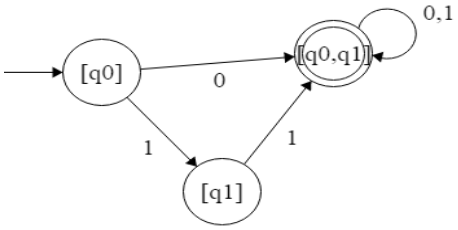
Step 4: Stop the procedure as there are no more new states being generated.

Step 5: The states which contain any of the final states of the NFA are the final states of the equivalent DFA.

q_1 is the final state in NFA. q_1 is included in the state [q_0,q_1] and [q_1]. So [q_0,q_1] and [q_1] are the final states of the DFA.

Q/∑	0	1
→ [q_0]	[q_0,q_1]	[q_1]
[q_0,q_1]	[q_0,q_1]	[q_0,q_1]
[q_1]	Ø	[q_0,q_1]

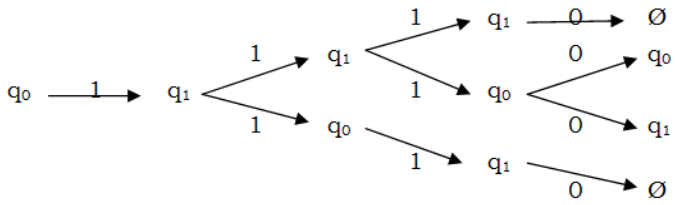
Step 6: Represent the transition diagram from the constructed table.



Step 7: To check the equivalence of NFA and DFA, the string accepted by NFA should be accepted by DFA.

Let **w=1110** be the string accepted by NFA.

Acceptability by NFA:



Acceptability by DFA:

$\delta([q_0],1110) = \delta([q_1],110)$

$= \delta([q_0,q_1],10)$

$= \delta([q_0,q_1],0)$

$= [q_0,q_1] \in F$

Step 8: Write the tuple representation from the obtained DFA.

DFA M' = $(Q,\sum, \delta,q_0,F)$

where $Q = \{[q_0], [q_0,q_1], [q_1]\}$

$\sum = \{0, 1\}$

δ - transition function

$[q_0]$ - initial state

$F = \{[q_0], [q_0,q_1]\}$

1.10 NFA with ε transitions:

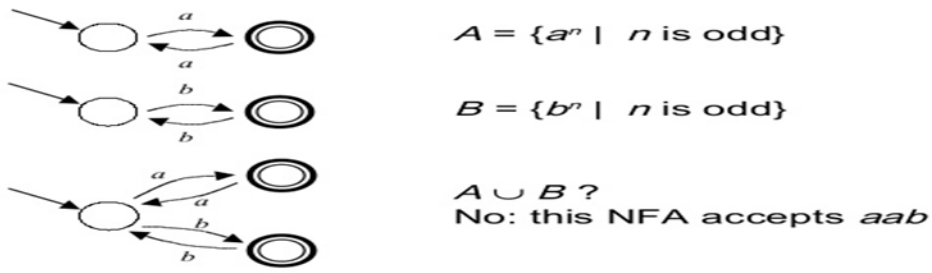
An ε -NFA is a tuple (Q, Σ, δ, qo, F) where

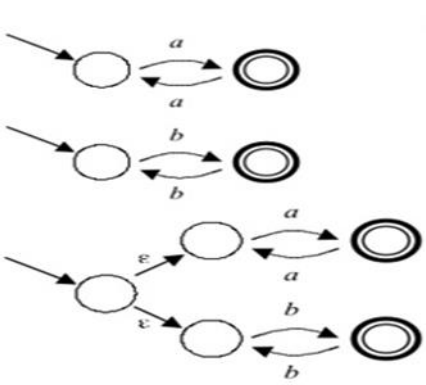
- Q is a set of states,
- Σ is the alphabet,
- δ is the transition function that maps each pair consisting of a state and a symbol in Σ ∪ { ε } to a subset of Q,
- q_0 is the initial state,
- F ⊂ Q is the set of final (or accepting) states.

1.11 Significance of ε-NFA:

It becomes very difficult or many times it seems to be impossible to draw directly NFA or DFA.

Example:





1.12 Minimization of FSM:

Two states ql and q*2* are equivalent (denoted by q1 ≡ *q2)* if both δ*(q1, x)* and δ*(q2, x)* are final states. or both of them are nonfinal states for all $x \in \sum^*$.

Two states q1 and *q2* are k-equivalent *(k* ≥ 0) if both δ*(q1, x)* and δ*(q2, x)* are final states or both nonfinal states for all strings *x* of length *k* or less. In particular, any two final states are 0-equivalent and any two nonfinal states are also 0-equivalent.

Construction of Minimum Automaton:

Step 1: (Construction of π_0)· By definition of 0-equivalence, $\pi_0 = \{Q_1^0, Q_2^0\}$ where Q_1^0 is the set of all final states and $Q_2^0 = Q - Q_1^0$.

Step 2: (Construction of π_{k+1} from π_k).

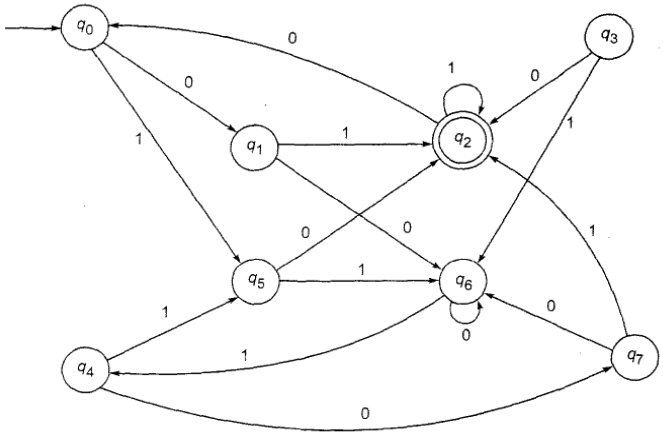
- Let Q_i^k be any subset in π_k . If q_1 and q_2 are in Q_i^k , they are (k + 1)-equivalent provided δ (q1,a) and δ(q2,a) are k-equivalent.
- Find out whether δ (q1, a) and δ (q2, a) are in the same equivalence class in π_k for every a ∈ ∑. If so q1 and q2 are (k + 1)-equivalent.
- In this way, Q_i^k is further divided into (k + 1)-equivalence classes. Repeat this for every Q_i^k in π_k to get all the elements of π_{k+1}.

Step 3: Construct π_n for n = 1, 2, ... until $\pi_n = \pi_{n+1}$.

Step 4: (Construction of minimum automaton). For the required minimum state automaton, the states are the equivalence classes obtained in step 3. i.e. the elements of π_n The state table is obtained by replacing a state q by the corresponding equivalence class [q].

Example:

Construct a minimum state automaton equivalent to the finite automaton.



Solution:

It will be easier if we construct the transition table.

State/Σ	0	1
→q_0	q_1	q_5
q_1	q_6	q_2
(q_2)	q_0	q_2
q_3	q_2	q_6
q_4	q_7	q_5
q_5	q_2	q_6
q_6	q_6	q_4
q_7	q_6	q_2

Step 1: Construction of π_0

$\pi_0 = \{Q_1^0, Q_2^0\}$

where $Q_1^0 = F = \{q2\}$ $Q_2^0 = Q - Q_1^0$

$\therefore \pi_0 = \{\{q2\}, \{q0,q1,q3,q4,q5,q6,q7\}\}$

Step 2: The {q2} in π_0 cannot be further partitioned. So, $Q_1^1 = \{q2\}$.

Compare q_0 with q_1, q_3,q_4,q_5,q_6 and q_7.

Consider qo and q1 ϵ Q_2^0.

- The entries under the 0- column corresponding to qo and q1 are q1 and q6; they lie in Q_2^0.
- The entries under the 1-column are q_5 and q_2. q2 ϵ Q_1^0 and q5 ϵ Q_2^0. Therefore qo and q1 are not 1- equivalent.

Q/∑	0	1
q_0	**q_1**	**q_5**
q_1	**q_6**	**q_2**

Consider q0 and q3

Q/∑	0	1
q_0	**q_1**	**q_5**
q_3	**q_2**	**q_6**

The entries under the 0- column corresponding to qo and q3 are q1 and q2; q1 ϵ Q_2^0 and q2 ϵ Q_1^0. The entries under the 1-column are q5 and q6; they lie in Q_2^0. Therefore qo and q3 are not 1-equivalent

Similarly, qo is not 1-equivalent to q5 and q7.

Consider q0 and q4

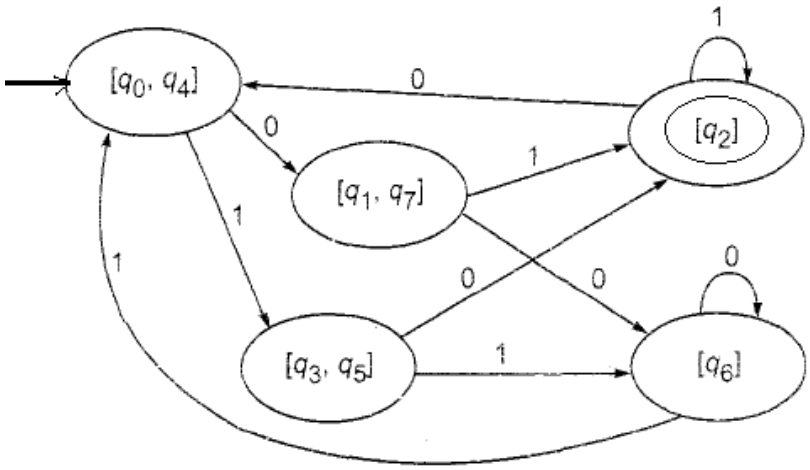
Q/∑	0	1
q0	q1	q5
q4	q7	q5

- The entries under the 0- column corresponding to qo and q4 are q1 and q7; they lie in Q_2^0.
- The entries under the 1-column are q5 and q5; they lie in Q_2^0. Therefore qo and q1 are 1- equivalent.
  Similarly, qo is 1-equivalent to q6.
  {qo. q4, q6} is a subset in π_1.
  So, Q_2^1 = {q0,q4,q6}
- Repeat the construction by considering q1 and anyone of the state's q3, q5, q7. Now, q1 is not 1-equivalent to q3 or q5 but 1-equivalent to q7.
  Hence, Q_3^1 = {q1,q7}.
- The elements left over in $Q_2^0$are q3 and q5. By considering the entries under the 0-column and the 1-column, we see that q3 and q5 are 1-equivalent.
  So Q_4^1 = {q3, q5}.
  Therefore, π_1 = {{q2}. {qo, q4, q6}. {q1, q7}, {q3, q5}}

Step 3: Construct π_n for n = 1, 2, ... until $\pi_n = \pi_{n+1}$.
Calculate 2-equivalent, π_2.
π_2 = {{q2}, {qo,q4}, {q6}, {q1,q7}, {q3,q5}}
Similarly calculate 3-equivalent, π_3.
π_3 = {{q2}, {qo,q4}, {q6}, {q1,q7},{q3,q5}}
As $\pi_2 = \pi_3$, π_2 gives us the equivalence classes.

Step 4: Construction of minimum automaton.
M' = (Q',{0,1},δ',q_0',F')
where Q' = {[q_2]. [q_0, q_4], [q_6], [q_1, q_7], [q_3, q_5]}
q_0' = [q0, q4]
F' = [q2]
δ' is given by

State/Σ	0	1
[q_0, q_4]	[q_1, q_7]	[q_3, q_5]
[q_1, q_7]	[q_6]	[q_2]
[q_2]	[q_0, q_4]	[q_2]
[q_3, q_5]	[q_2]	[q_6]
[q_6]	[q_6]	[q_0, q_4]



1.13 Moore Machine:

A Moore machine is a six tuple (Q, ∑, Δ, δ, q_0, λ) where

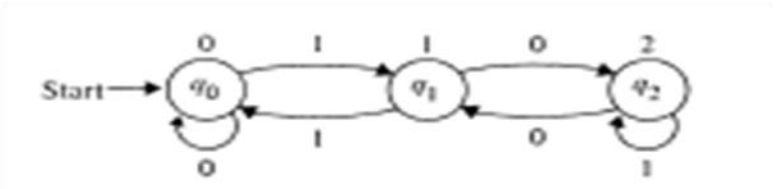
- Q is a set of states,
- Σ is the alphabet,
- δ is the transition function that maps each pair consisting of a state and a symbol in Σ to Q i.e. .Q X Σ -> Q
- q0 is the initial state,
- Δ is output alphabet
- λ is a mapping from Q to Δ giving the output associated with each state

Note: For a Moore machine if the input string is of length n, the output string is of length n + 1. The first output is λ (qo) for all output strings.

is also of the same length n.

Example:

- The given transition diagram is moore machine because each state is associated with output.
- In the below diagram q_0 is representing 0 output, q_1 is is representing 1 output and q_2 is representing 2 output.
  λ (q_0) =0 λ (q_1)=1 λ (q_2)=2



w=011 the output is **0010**

q_0 —0→ q_0 —1→ q_1 —1→ q_0 Transitions

0 0 1 0 Outputs

1.17 Mealy Machine:

A Mealy machine is a six tuple (Q, ∑, Δ, δ, q_0, λ) where

- Q is a set of states,
- Σ is the alphabet,
- δ is the transition function that maps each pair consisting of a state and a symbol in Σ to Q i.e. .Q X Σ -> Q
- Δ is output alphabet
- q0 is the initial state,
- **λ** maps Q x ∑ **to** Δ **i.e.,** λ(q,a) gives the output associated with the transition from state q on input a

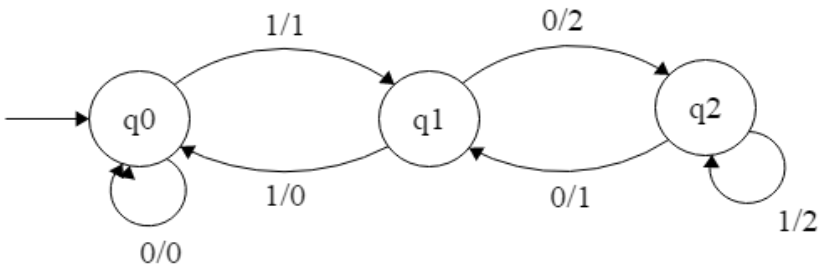
Note: In the case of a Mealy machine if the input string is of length n , the output string

Example:

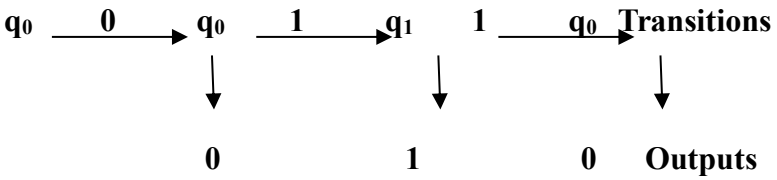
- The given transition diagram is mealy machine because output depends on present state and present input.
- In the below diagram

$\lambda(q_0,0) = 0$ $\lambda(q_1,0) = 2$ $\lambda(q_2,0) = 0$

$\lambda(q_0,1) = 1$ $\lambda(q_1,1) = 0$ $\lambda(q_2,1) = 2$

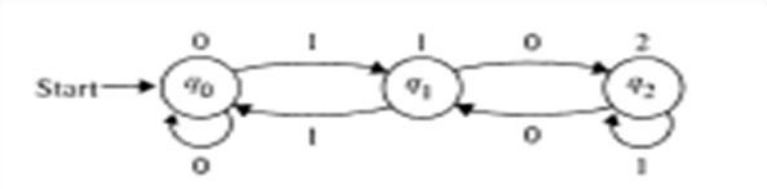


w=011 the output is **010**



Example:

1. Design Moore machine to determine the residue mod 3 for each binary string treated as a binary integer.



Moore machine calculating residue mod 3

Moore Table

	Present S	Next State		Outpu
		0	1	
→	q 0	q0	q1	0
	q1	q2	q0	1
	q2	q1	q2	2

Tuple Representation:

Q=$\{q_0,q_1,q_2\}$ $\Delta=\{0,1,2\}$ $\sum=\{0,1\}$

q_0=$\{q_0\}$

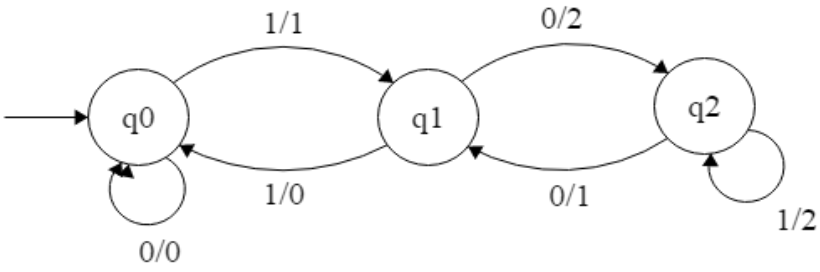
λ :λ $(q_0)=0$ **δ:** $\delta(q_0,0) = q_0$ $\delta(q_0,1) = q_1$

λ $(q_1)=1$ $\delta(q_1,0) = q_2$ $\delta(q_1,1) = q_0$

λ $(q_2)=2$ $\delta(q_2,0) = q_1$ $\delta(q_2,1) = q_2$

Example:

1. Design Mealy machine to determine the residue mod 3 for each binary string treated as a binary integer.



Mealy Table:

	Present S	Next State		Next State	
		0	Outpu	1	Outpu
→	q 0	q0	0	q1	1

q_1	q_2	2	q_0	0
q_2	q_1	1	q_2	2

Tuple Representation:

Q=$\{q_0,q_1,q_2\}$ $\Delta=\{0,1,2\}$ $\sum=\{0,1\}$

$\mathbf{q_0}=\{q_0\}$

λ: $\lambda(q_0,0)=0$, $\lambda(q0,1)=1$, $\lambda(q_1,0)=2$, $\lambda(q_1,1)=0$, $\lambda(q_2,0)=1$, $\lambda(q_2,1)=2$

δ: $\delta(q_0,0)=q_0$, $\delta(q_0,1)=q_1$, $\delta(q_1,0)=q_2$, $\delta(q_1,1)=q_0$, $\delta(q_2,0)=q_1$, $\delta(q_2,1)=q_2$

1.14 Differences between NFA and DFA:

S.No	NFA	DFA
1	A nondeterministic finite automaton is tuple **M= (Q, ∑, δ, q_0, F)**, where **δ**: Q x ∑ into 2^Q.	A deterministic finite automaton can represented by a 5-tuple **M= (Q, ∑, δ, q_0, F),** where **δ**: Q x ∑ to Q.
2	NFA is the one in which there exists paths for a specific input from current to next state.	DFA is a FA in which there is only one path specific input from current state to next state.
3	NFA is easier to construct.	DFA is more difficult to construct.
4	NFA requires less space.	DFA requires more space.
5	Time required for executing an input is more.	Time required for executing an input string is

1.15 Applications and Limitation of Finite Automata

Applications of Finite Automata:

1. **Lexical Analysis in Compilers**
   - Used to recognize tokens like identifiers, keywords, operators, etc.
   - Implemented using **Deterministic Finite Automata (DFA)**.
2. **Pattern Matching**
   - Useful in tools like grep, awk, and sed for regular expression matching.
3. **Text Search Algorithms**
   - Algorithms like Knuth-Morris-Pratt (KMP) use FAs for efficient searching.
4. **Protocol Design and Verification**
   - Communication protocols are modeled as finite state machines to verify correctness.
5. **Digital Circuit Design**
   - Used in designing sequential circuits, where input sequences control outputs via states.
6. **Modeling Control Systems**
   - Used to simulate and control logical sequences in systems like elevators, vending machines, etc.

7. **Natural Language Processing (NLP)**
   - Used for simple syntax checking and token classification.
8. **Network Intrusion Detection Systems (NIDS)**
   - Match known attack signatures using regular expressions (converted into finite automata).

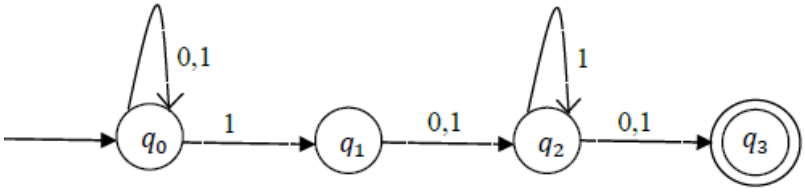
Limitations of Finite Automata:

1. **No Memory of Past Input (Beyond Current State)**
   - FA cannot remember complex input histories or count occurrences (e.g., cannot recognize a^n b^n).
2. **Cannot Recognize Non-Regular Languages**
   - Languages that require matching nested structures (like parentheses) or counting cannot be recognized.
3. **No Stack or Tape**
   - Unlike Pushdown Automata (PDA) or Turing Machines, FA lacks auxiliary memory or data structures.
4. **Static State Set**
   - Fixed number of states may make representation of complex logic inefficient or impractical.
5. **Non-scalable for Context-Sensitive Tasks**
   - FA is not suitable for parsing context-free or context-sensitive languages.

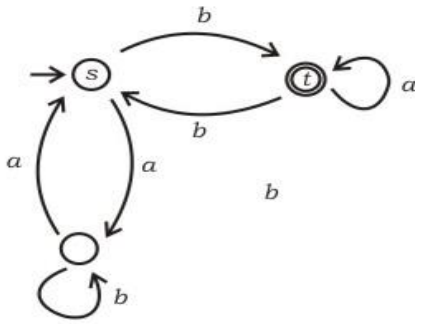
Assignment-Cum-Tutorial Questions

Part-A
Objective Questions (1M)

1. The prefix of abc is [ ]
   A) c B) bc C) b D) ε
2. $\sum^* = \sum^+ U \varepsilon$ [True | False]
3. Alphabet is _____. [ ]
   A) Finite collection of strings. B) Finite collection of symbols.
   C) Finite collection of languages. D) All the above.
4. A _____ of a string S is any trailing contiguous part of symbols of S.
5. _____ is a directed graph associated with an FA in which the vertices of the graph correspond to the states of the FA.
6. The transition function for NFA is a mapping function given as _____.
7. The transition function for DFA is a mapping function given as _____.
8. A = {a, b, c}. Power set of A= _____
9. FA has [ ]
   A) Unlimited memory B) no memory at all
   C) Limited memory D) none of the above.
10. Number of states requires to accept string ends with 10. [ ]
    A) 3 B) 2 C) 1 D) can't be represented.
11. Consider the finite automaton in the following figure



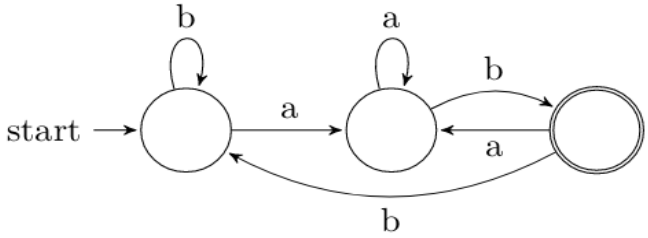
    What is the set of reachable states for the input string 0011? [ ]
    A) $\{q_0, q_1, q_2\}$ B) {q0, q1} C) {q0, q1, q2, q3} (D) {q3}
12. Given the language L = {ab, aa, baa}, which of the following strings are in L*?
    1) abaabaaabaa 2) aaaabaaaa
    3) baaaaabaaaab 4) baaaaabaa [ ]
    A) 1,2 and 3 B) 2,3 and 4 C) 1,2 and 4 D) 1, 3 and 4
13. In the automaton below, s is the start state and t is the only final state.



Consider the strings u = abbaba, v = bab, and w = aabb. Which of the following statements is true? [ ]

A) The automaton accepts u and v but not w
B) The automaton accepts each of u,v and w
C) The automaton rejects each of u,v and w
D) The automaton accepts u but rejects v and w

14. If the final states and non-final states in the DFA below are interchanged, then which of the following languages over the alphabet {a,b} will be accepted by the new DFA? [ ]

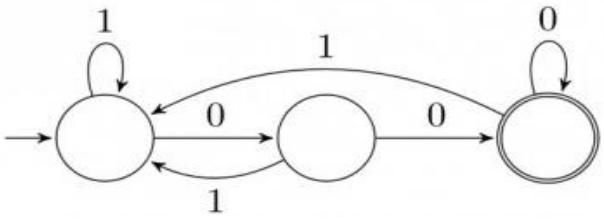


A) Set of all strings that do not end with ab
B) Set of all strings that begin with either an a or a b
C) Set of all strings that do not contain the substring ab,
D) All the above

15. The smallest finite automation which accepts the language {x | length of x is divisible by 3} has [ ]

A) 2 states B) 3 states C) 4 states D) 5 states

16. The below DFA accepts the set of all strings over {0,1} that [ ]



A) begin either with 0 or 1 B) end with 0
C) end with 00 D) contain the substring 00

17. Consider a DFA over ∑={a,b} accepting all strings which have number of a's divisible by 6 and number of b's divisible by 8. What is the number of states that the DFA will have? [ ]

A) 8 B) 14 C) 15 D) 48

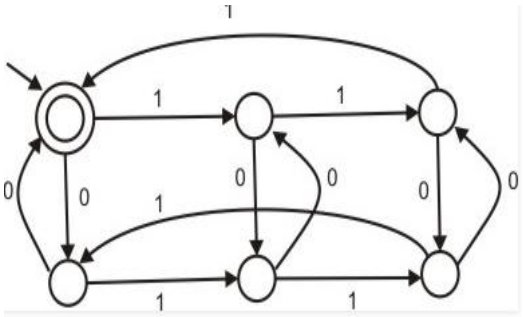
18. What is the minimum number of states in the NFA accepting the language {a, ab} ? [ ]

A) 3 B) 2 C)1 D) 4

19. What is the number of states in NFA which accepts set of all strings in which the third last symbol is 'a' over alphabet {a, b}? [ ]

A) three B) four C) six D) five

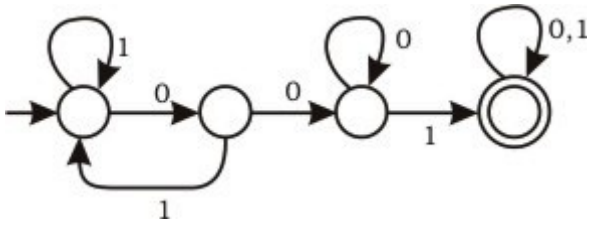
20. The following finite state machine accepts all those binary strings in which the number of 1's and 0's are respectively [ ]



A) divisible by 3 and 2 B) odd and even

C) even and odd D) divisible by 2 and 3

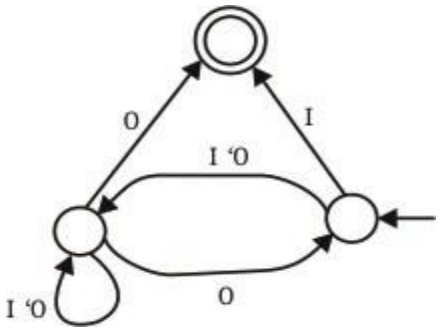
21. Consider the following deterministic finite state automaton M.



Let S denote the set of seven bit binary strings in which the first, the fourth, and the last bits are 1. The number of strings in S that are accepted by M is

A) 1 B) 5 C) 7 D) 8 [ ]

22. Consider the NFA M shown below.



Let the language accepted by M be L. Let L1 be the language accepted by the NFA M1, obtained by changing the accepting state of M to a non-accepting state and by changing the non-accepting state of M to accepting states. Which of the following Statements is true? [ ]

A) L1 = {0,1}* – L B) L1={0,1}* C) L1 ⊆ L D) L1 = L

23. Construct a finite state machine that accepts the language, over {0,1},of all strings that contain neither the substring 00 nor the substring 11.

24. What can be said about a regular language L over {a} whose minimal finite state automaton has two states? [ ]

A) L must be $\{a^n \mid n \text{ is odd}\}$
B) L must be $\{a^n \mid n \text{ is even}\}$
C) L must be $\{a^n \mid >=0\}$
D) Either L must be $\{a^n \mid n \text{ is odd}\}$, or L must be $\{a^n \mid n \text{ is even}\}$

Part-B

Short Answer Questions (2M)

1. Define (i) String (ii) Alphabet (iii) Language (iv) Closure (v) Powerset
2. Explain in detail various operations performed on Strings and Languages.
3. Define i) Positive Closure ii) Kleene Closure.
4. Define Finite Automaton(FA). Explain in detail different ways to represent it.
5. Sketch and explain in detail the model of FA. Give its tuple notation.
6. Differentiate between NFA and DFA.
7. What is the difference between empty language and null string?
8. Which of the following Finite Automaton is having ambiguity and why?
   i) NFA ii) DFA
9. Draw the Finite state machine for accepting the languages Ɛ and Ø.
10. Draw Transition diagram for the Transition table given below and check acceptance for the strings:
    i) 1000101 ii) 011001

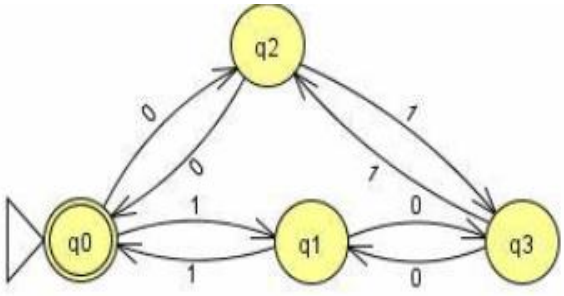
	0	1
→ q0	q1	q0
q1	q2	q0
q2	q3	q0
(q3)	q3	q3

11. Draw the transition diagram for below FA:

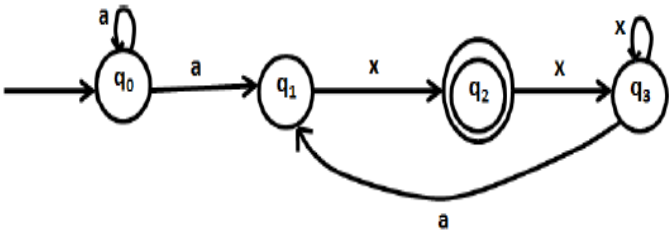
M= { {A,B,C,D}, {0,1}, δ,C, {A,C} }
δ (A,0) = δ (A,1) = {A,B,C}
δ(B,0) = B, δ(B,1) = { A, C }
δ(C,0) = {B,C}, δ(C,1) ={ B, D }
δ (D,0) = { A, B, C, D }
δ(D,1) = {A}.

Part C
Descriptive Questions (5M)

1. Construct DFA accepting the set of all strings beginning with 101.
2. Design a DFA for a language which contains strings of a's & b's and each string ends with aab.
3. Describe the words w in the language L accepted by the automaton in



4. Design a DFA that accepts set of all strings with abab as a substring over alphabet {a, b}.
5. Design a DFA that accepts set of all strings with three consecutive 0's over alphabet {0, 1}.
6. Design a DFA that accepts all strings over Σ={0,1} that do not contain 101 as a substring.
7. Obtain a DFA to accept strings of a's and b's such that, each block of 5 consecutive symbols has at least two a's.
8. a) Design a DFA to accept the following language. L = {w: |w| mod 3 = 0} on Σ = {a}
   b) Design DFA accepting the language whose binary interpretation is divisible by 5 over the alphabet {0, 1}.
9. Design a DFA that accepts set of all strings in which number of a's is divisible by 4 over alphabet {0, 1}.
10. Design a DFA to accept strings of a's and b's having even number of a's and b's.
11. For the NFA given below;
    i. Check whether the string axxaxxa is accepted or not
    ii. Give atleast two transition paths



12. Design DFA accepting the set of all strings that begin with 01 and end with 11.
13. Design NFA accepting the set of all strings that begins with abb over alphabet {a, b}.
14. Design a NFA that accepts set of all strings with aba as a substring over alphabet {a, b}.

15. Design a NFA to accept strings of 0's & 1's such that each string ends with 00.
16. Construct DFA equivalent to the NFA given below

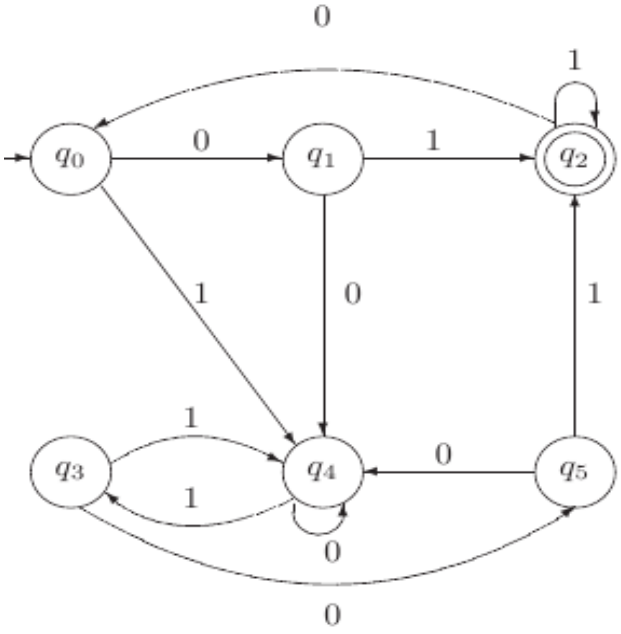
M=({q0,q1,q2,q3},{a,b}, δ , q0, {q3}) where δ is given as

	a	b
→ q0	{q0,q1}	q0
q1	q2	q1
q2	q3	q3
(q3)	-	q2

17. Construct an equivalent DFA for a NDFA M = ({q1, q2, q3}, q1, q3) where δ is given by:

δ(q1,0)={ q2,q3 }, δ(q1,1)={ q1 },
δ(q2,0)={ q1,q2 }, δ(q2,1)= ∅
δ(q3,0)={ q2} , δ(q3,1)={ q1,q2 }

18. Construct the minimum state automaton equivalent to the transition diagram



19. Construct a minimum state automaton equivalent to a given automaton M whose transition table is defined by table

	0	1
→ A	B	D
B	C	E
C	B	E
D	C	E
(E)	E	E

20. Design a Moore machine to determine the residue mod 4 for each binary string treated as integers.
21. Construct a Mealy machine which can output EVEN or ODD according as the total number of 1's encountered is even or odd. The input symbols are 0 and 1.
22. Design a Mealy machine to find out 2's complement of a binary number.
23. Design Moore and Mealy machines to find out 1's complement of a binary number.
24. Construct a Moore machine that takes set of all strings over {a,b} as input and prints '1' as output for every occurrence of 'ab' as a substring.
25. Design a Mealy machine that uses its states to remember the last symbol read and emits output 'y' whenever current input matches to previous one, and emits n otherwise
26. Give Mealy and Moore machines for the following process: For input from (0+1)*, if the input ends in 101, output A; If the input ends in 110 output B; otherwise output C.
27. Convert the following Mealy machine to Moore machine

Present State	Next State			
	a = 0		a = 1	
	State	Output	State	Output
-> q0	q3	0	q1	1
q1	q0	1	q3	0
q2	q2	1	q2	0
q3	q1	0	q0	1

28. Construct a Mealy machine which is equivalent to the Moore machine given in table:

Present State	Next State		Output
	A=0	A=1	
→ q_0	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

29. Convert the following Mealy machine into its equivalent Moore machine

